CS-E4740 - Federated Learning

L4 - FL Algorithms II

Dipl.-Ing. Dr.techn. Alexander Jung

Spring 2026

Calendar **Glossary** **Book** **GitHub**

Table of Contents

FL as Fixed-Point Iterations

Applying Gradient Methods to GTVMin

Synchronous FL Algorithms

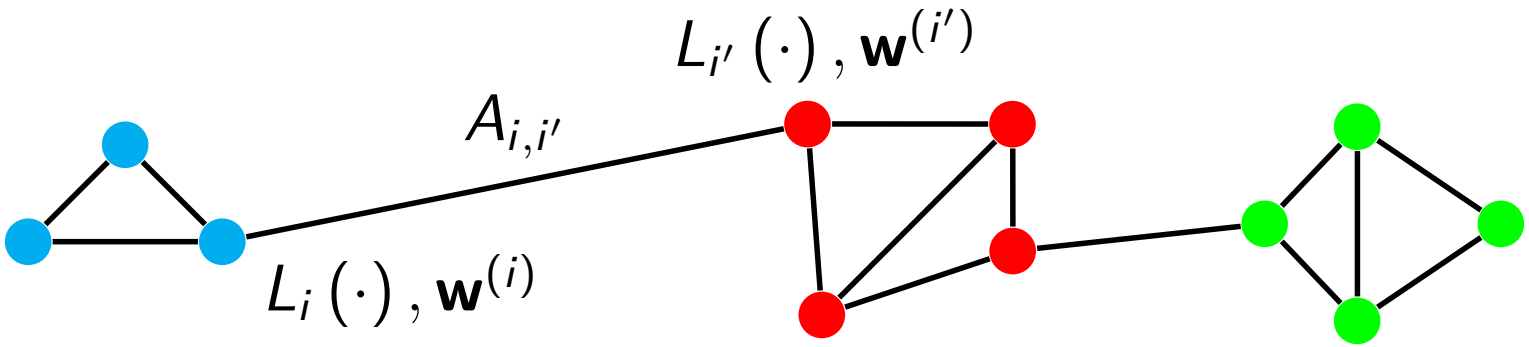
Table of Contents

FL as Fixed-Point Iterations

Applying Gradient Methods to GTVMin

Synchronous FL Algorithms

Recap: Federated learning network (FL network)



- FL network consists of devices $i = 1, \ldots, n$
- undirected edges $\{i, i'\} \in \mathcal{E}$ with weight $A_{i,i'} > 0$
- device i learns model parameters $\mathbf{w}^{(i)} \in \mathbb{R}^d$, e.g.

$$h^{(\mathbf{w}^{(i)})}(\mathbf{x}) := \mathbf{x}^T \mathbf{w}^{(i)}$$

- usefulness of $\mathbf{w}^{(i)}$ measured by device-specific loss $L_i()$

Recap: Our FL Design Principle

couple local model training at devices via GTVMin

$$\min_{\substack{h^{(1)} \in \mathcal{H}^{(1)} \\ \cdots \\ h^{(n)} \in \mathcal{H}^{(n)}}} \sum_{i=1}^{n} \underbrace{L_i\left(h^{(i)}\right)}_{\text{train. at } i} + \alpha \sum_{\{i,i'\} \in \mathcal{E}} A_{i,i'} \underbrace{d^{(h^{(i)}, h^{(i')})}}_{\text{discrepancy betw. } i,i'} \qquad \text{(GTVMin)}$$

parameter α controls clustering of trained local models

this lecture presents FL algorithms which solve GTVMin via fixed-point iterations

Recap: Fixed-point characterization

necessary condition for GTVMin solution $\widehat{h}^{(i)}$

$$\widehat{h}^{(i)} = \mathcal{F}^{(i)}\left(\widehat{h}^{(1)}, \ldots, \widehat{h}^{(n)}\right), \text{ for } i = 1, \ldots, n \quad \text{(NecCond)}$$

with the operator

$$\mathcal{F}^{(i)}\left(\widehat{h}^{(1)}, \ldots, \widehat{h}^{(n)}\right) := \underset{h \in \mathcal{H}^{(i)}}{\arg\min}\, L_i\left(h\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} d^{(h, \widehat{h}^{(i')})}$$

note that

- $\mathcal{F}^{(i)}$ depends on local dataset via $L_i\left(\cdot\right)$
- $\mathcal{F}^{(i)}$ depends on FL network via $\mathcal{N}^{(i)}$ and $A_{i,i'}$
- the operator $\mathcal{F}^{(i)}$ in (NecCond) can be chosen differently

A Blueprint for Federated learning (FL) Algorithms

- assume we found a fixed-point characterization of GTVMin with operators $\mathcal{F}^{(i)}$
- these operators induce an iterative algorithm
- during iteration t, each device i computes

$$\widehat{h}^{(i,t+1)} = \mathcal{F}^{(i)}\big(\widehat{h}^{(1,t)}, \ldots, \widehat{h}^{(n,t)}\big) \quad \text{(FLAlgo)}$$

- NOTE: update is done synchronously by all devices

Example: FL for Parametric models

GTVMin instance

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}\in\mathbb{R}^d} \sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2$$

any solution is a fixed point of

$$\mathcal{F}^{(i)}\left(\widehat{\mathbf{w}}^{(1)},\ldots,\widehat{\mathbf{w}}^{(n)}\right) := \operatorname*{arg\,min}_{\mathbf{w}^{(i)}\in\mathbb{R}^d} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \widehat{\mathbf{w}}^{(i')}\right\|_2^2$$

other choices $\mathcal{F}^{(i)}$ also allow fixed-point characterization

Table of Contents

FL as Fixed-Point Iterations

Applying Gradient Methods to GTVMin

Synchronous FL Algorithms

Gradient-based methods

consider optimization problem $\min_{\mathbf{w}} f(\mathbf{w})$ with smooth and convex objective function f

- solution $\mathbf{w}^\star$ characterized by zero-gradient condition

$$\nabla f(\mathbf{w}^\star) = \mathbf{0}$$

- this condition is equivalent to

$$\mathcal{F}^{(f,\eta)}\mathbf{w}^\star = \mathbf{w}^\star$$

with the gradient step $\mathcal{F}^{(f,\eta)}\mathbf{w} := \mathbf{w} - \eta\nabla f(\mathbf{w})$

Gradient step for GTVMin

- consider GTVMin with smooth and convex $L_i(\cdot)$,

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}\in\mathbb{R}^d} \sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2 \qquad (1)$$

- stack GTVMin solution $\widehat{\mathbf{w}}^{(i)} \in \mathbb{R}^d$, for $i = 1, \ldots, n$ into

$$\widehat{\mathbf{w}} := \left(\widehat{\mathbf{w}}^{(1)}, \ldots, \widehat{\mathbf{w}}^{(n)}\right)^T \in \mathbb{R}^{dn}$$

- $\mathcal{F}^{(\eta)}$ maps $\mathbf{u} = \left(\mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(n)}\right)^T$ to $\mathbf{v} = \left(\mathbf{v}^{(1)}, \ldots, \mathbf{v}^{(n)}\right)^T$,

$$\mathbf{v}^{(i)} = \mathbf{u}^{(i)} - \eta\left[\nabla L_i\left(\mathbf{u}^{(i)}\right) + 2\alpha \sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'}\left(\mathbf{u}^{(i)} - \mathbf{u}^{(i')}\right)\right].$$

$$\boxed{\widehat{\mathbf{w}} \text{ solves } (1) \iff \widehat{\mathbf{w}} = \mathcal{F}^{(\eta)}\widehat{\mathbf{w}} \quad \text{for any step size } \eta > 0}$$

Interpretation of Gradient step

start from initial model parameters $\mathbf{w}^{(i,0)}$, repeat gradient steps

$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta_{t,i}\left[\nabla L_i\left(\mathbf{w}^{(i,t)}\right) + 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'}\left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)}\right)\right]$$

- learning rate $\eta_{t,i}$ determines extent of update
- $\nabla L_i\left(\mathbf{w}^{(i,t)}\right)$ **pulls** $\mathbf{w}^{(i,t)}$ towards **min. local loss**
- $\left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)}\right)$ **pulls** $\mathbf{w}^{(i,t)}$ to **agree with neighbors**
- $\alpha A_{i,i'}$ balances those two steering effects

Synchronous Operation

gradient step

$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta_{t,i} \left[\nabla L_i \left(\mathbf{w}^{(i,t)} \right) + 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)} \right) \right]$$

carried out by all nodes $i = 1, \ldots, n$ "simultaneously"

after completing (local) gradient step, each node shares new model parameters with their neighbors

after sharing model parameters, start new iteration $t := t+1$

Message Passing Implementation

consider two connected nodes $i, i' \in \mathcal{V}$ of a FL network

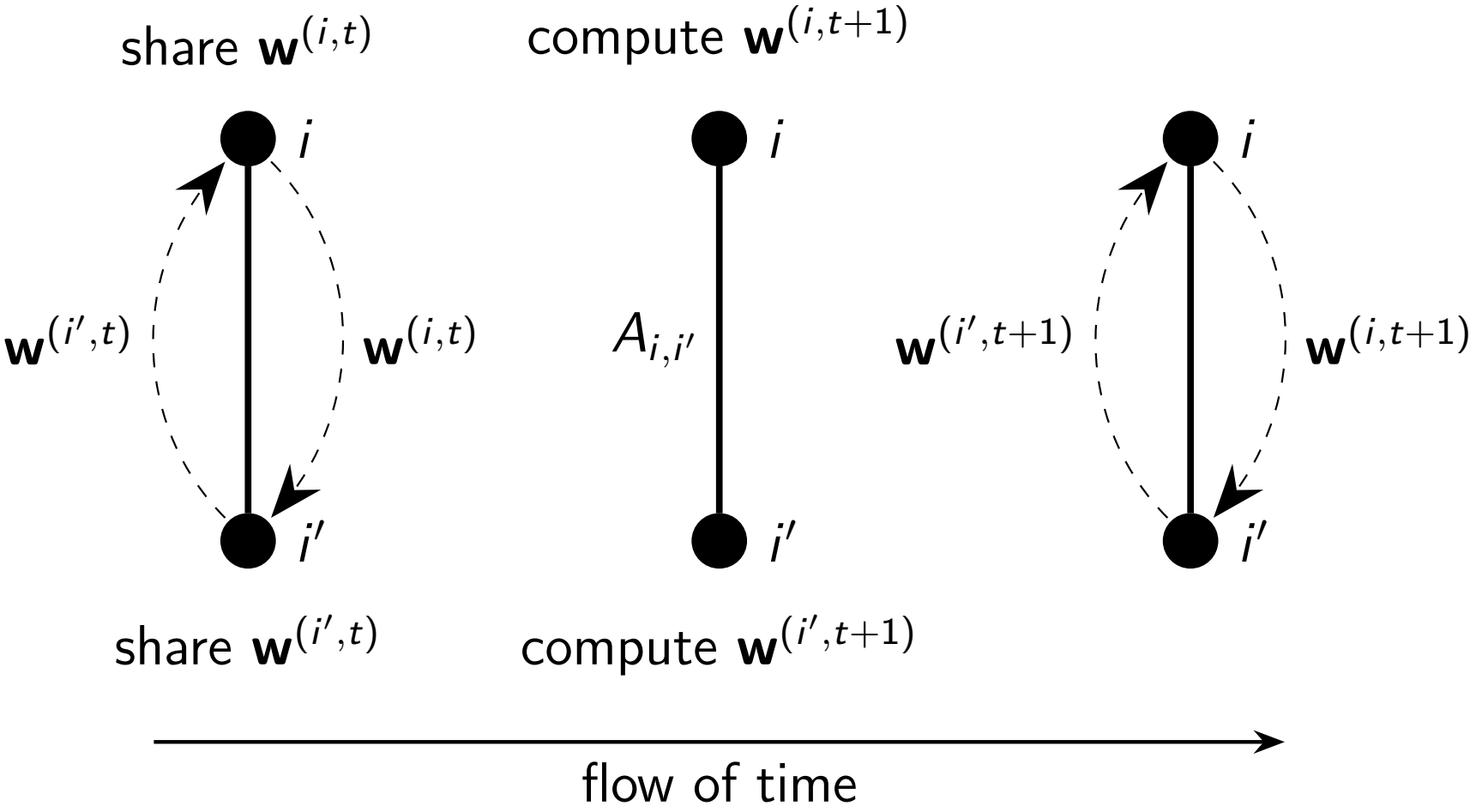


Table of Contents

FL as Fixed-Point Iterations

Applying Gradient Methods to GTVMin

Synchronous FL Algorithms

Federated GD (FedGD)

each node $i = 1, \ldots, n$ initializes

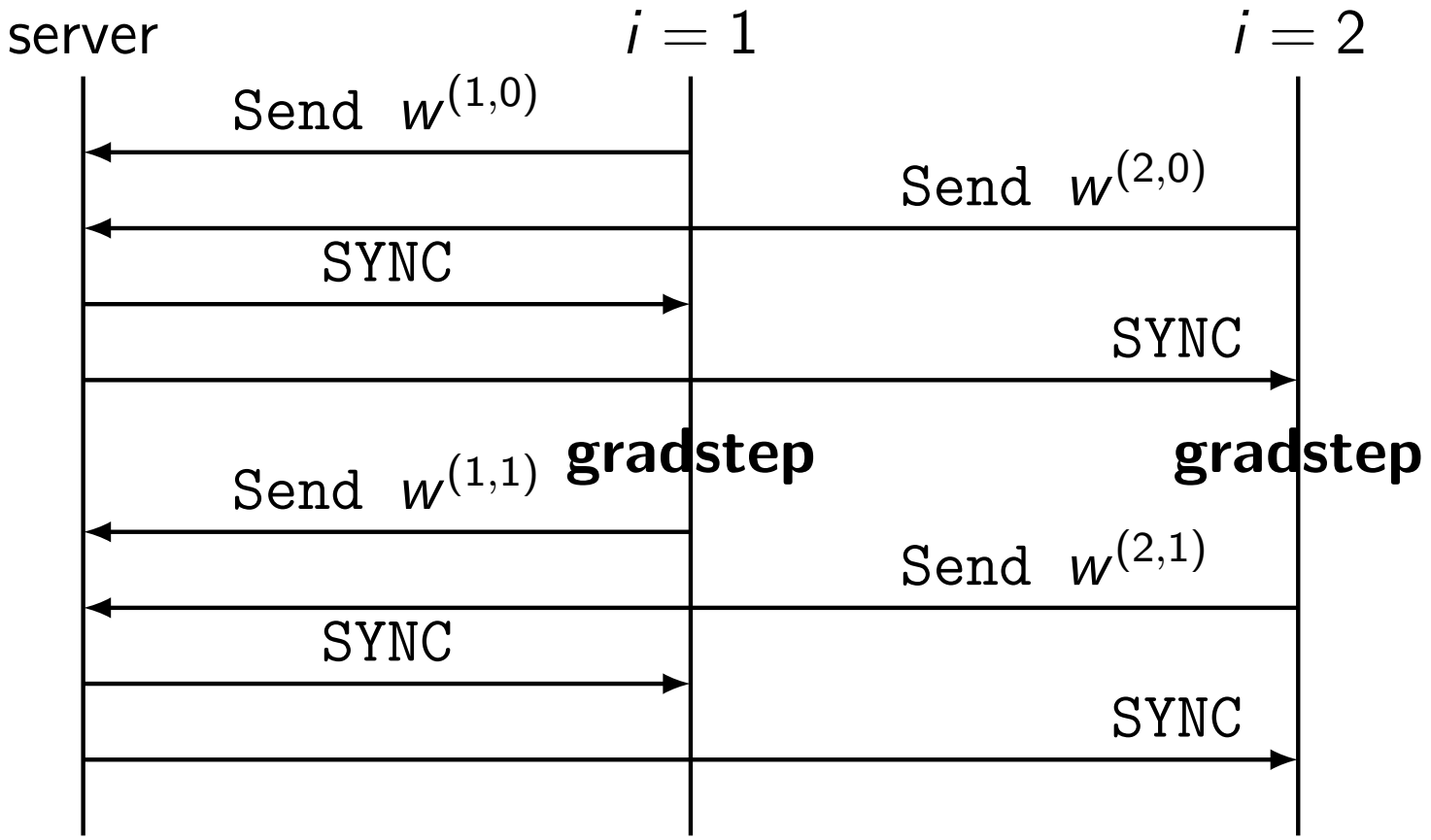
- local model parameters $\mathbf{w}^{(i,0)} := \mathbf{0}$, and
- iteration counter $t := 0$.

repeat the following steps at each node i:

- send $\mathbf{w}^{(i,t)}$ to all neighbors $\mathcal{N}^{(i)}$
- do a gradient step
- increment iteration counter $t := t + 1$

CAUTION: nodes must execute steps synchronously!

Implementing FedGD with a Sync-Server



Python demo

Federated Stochastic Gradient Descent (FedSGD)

- consider FL network with node i carrying the local dataset

$$\mathcal{D}^{(i)} = \left\{ \left(\mathbf{x}^{(i,1)}, y^{(i,1)}\right), \ldots, \left(\mathbf{x}^{(i,m_i)}, y^{(i,m_i)}\right) \right\}$$

- Node i uses local loss function

$$L_i\left(\mathbf{w}^{(i)}\right) := (1/m_i) \sum_{r=1}^{m_i} \left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T \mathbf{x}^{(i,r)} \right)^2$$

- FedGD requires to compute gradient,

$$\nabla L_i\left(\mathbf{w}^{(i)}\right) = (-2/m_i) \sum_{r=1}^{m_i} \mathbf{x}^{(i,r)} \left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T \mathbf{x}^{(i,r)} \right)$$

Stochastic Gradient Approximation

the computation of

$$\sum_{r=1}^{m_i} \mathbf{x}^{(i,r)} \left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T \mathbf{x}^{(i,r)} \right)$$

can be infeasible in practice, e.g., too many data points or access too slow

$\Rightarrow$ use instead a sum over random subset $\mathcal{B}^{(i)} \subseteq \{1, \ldots, m_i\}$,

$$\sum_{r \in \mathcal{B}^{(i)}} \mathbf{x}^{(i,r)} \left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T \mathbf{x}^{(i,r)} \right)$$

we refer to $\mathcal{B}^{(i)}$ as a batch

FedAvg

- some FL applications use common model at all nodes,

$$\mathbf{w}^{(i)} = \mathbf{w}^{(i')} \quad \forall\, i, i' \in \mathcal{V}.$$

- GTVMin becomes constrained optimization problem:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \sum_{i \in \mathcal{V}} L_i\left(\mathbf{w}^{(i)}\right) \text{ s.t. } \quad \mathbf{w}^{(i)} = \mathbf{w}^{(i')} \quad \forall\, i, i' \in \mathcal{V}.$$

- for differentiable $L_i\left(\mathbf{w}^{(i)}\right)$ we can apply projected GD.

- projection step amounts to averaging $(1/n)\sum_{i=1}^{n} \mathbf{w}^{(i)}$.

(Almost) FedAvg

Init. counter (clock) $t := 0$ and model parameters $\widehat{\mathbf{w}} := \mathbf{0}$.

1. **Broadcast.** server sends $\widehat{\mathbf{w}}$ to all nodes $i \in \mathcal{V}$
2. **Local gradient step.** each node computes

$$\mathbf{w}^{(i,t)} = \widehat{\mathbf{w}} - \eta_{t,i} \nabla L_i \left(\widehat{\mathbf{w}} \right)$$

3. **Collect.** nodes send $\mathbf{w}^{(i,t)}$ back to server
4. **Aggregate.** server computes $\widehat{\mathbf{w}} := (1/n) \sum_{i=1}^{n} \mathbf{w}^{(i,t)}$
5. **Clock tick.** server increments $t := t + 1$; go to step 1

FedAvg

we obtain FedAvg via the following modifications:

- use (stochastic) approximations of gradients
- instead of single gradient step, compute several gradient steps
- each iteration involves only a subset of nodes

B. McMahan et.al., Communication-Efficient Learning of Deep Networks from Decentralized Data, PMLR, 2017

FedProx

replace gradient steps in FedAvg with proximal operator

$$\mathbf{w}^{(i)} := \arg\min_{\mathbf{v}\in\mathbb{R}^d} \left[L_i(\mathbf{v}) + (1/\eta)\|\mathbf{v} - \widehat{\mathbf{w}}\|_2^2 \right]$$

here, $\widehat{\mathbf{w}}$ denotes current global model parameters

compared with FedAvg, FedProx has been observed to

- improve robustness of FL system
- require less tuning when devices are heterogeneous

T. Li, et.al, Federated Optimization in Heterogeneous Networks, Proc. of Machine Learning and Systems 2, 2020.

Federated Relaxation (FedRelax)

- consider GTVMin objective function

$$f\left(\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}\right)=\sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right)+\alpha\sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'}\left\|\mathbf{w}^{(i)}-\mathbf{w}^{(i')}\right\|_2^2$$

- complicated due to coupling terms $A_{i,i'}\left\|\mathbf{w}^{(i)}-\mathbf{w}^{(i')}\right\|_2^2$
- without coupling, GTVMin would be much easier
- optimize $f(\cdot)$ w.r.t. $\mathbf{w}^{(i)}$, holding $\left\{\mathbf{w}^{(i')}\right\}_{i'\in\mathcal{V}\setminus\{i\}}$ fixed!

Similar to Jacobi method for solving linear equations. See Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., 2003, Ch. 4.1.

FedRelax for Parametric Models

- set counter $t := 0$, local model parameters $\widehat{\mathbf{w}}^{(i,0)} := \mathbf{0}$
- **repeat until stopping criterion:**
  - **Share.** each node i shares $\widehat{\mathbf{w}}^{(i,t)}$ with neighbors $\mathcal{N}^{(i)}$
  - **Local Update.** each node i computes

$$\mathbf{w}^{(i,t+1)} := \underset{\mathbf{w}^{(i)} \in \mathbb{R}^d}{\arg\min}\, L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left\| \mathbf{w}^{(i)} - \mathbf{w}^{(i',t)} \right\|_2^2$$

  - **Clock Tick.** $t := t+1$

Note the similarity of local update with ridge regression.

FedRelax for Non-Parametric Models

- **Init.** $t := 0$, construct test-set $\mathcal{D}^{\{i,i'\}}$ for each $\{i, i'\} \in \mathcal{E}$
- **Repeat until stopping criterion:**
  - **Share.** each node i shares predictions $\widehat{h}_t^{(i)}(\mathbf{x})$ on test-set $\mathbf{x} \in \mathcal{D}^{\{i,i'\}}$ with each neighbor $i' \in \mathcal{N}^{(i)}$
  - **Local Update.** each node i computes

$$\widehat{h}_{t+1}^{(i)} \in \operatorname*{arg\,min}_{h^{(i)} \in \mathcal{H}^{(i)}} L_i\left(h^{(i)}\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} d^{(h^{(i)}, \widehat{h}_t^{(i')})}$$

  - **Clock Tick.** $t := t + 1$

example for discrepancy measure

$$d^{(h^{(i)}, h^{(i')})} := (1/m') \sum_{\mathbf{x} \in \mathcal{D}^{\{i,i'\}}} \left[h^{(i)}(\mathbf{x}) - h^{(i')}(\mathbf{x})\right]^2$$

Table of Contents

Asynchronous FL Algorithms

FL as Fixed-point iterations

- consider FL network with parametric models
- synchronous FL algorithms consist of updates

$$\mathbf{w}^{(i,t+1)} = \mathcal{F}^{(i)}\left(\mathbf{w}^{(1,t)}, \ldots, \mathbf{w}^{(n,t)}\right)$$

  with node-specific operators

$$\mathcal{F}^{(i)} : \mathbb{R}^{nd} \to \mathbb{R}^{d} \text{ , for each node } i = 1, \ldots, n$$

- FL algorithm determined by the local update operators $\mathcal{F}^{(1)}, \ldots, \mathcal{F}^{(n)}$

Local updates can be time-varying, i.e., using $\mathcal{F}^{(i,t)}$ varying with t.

Challenges of Synchronous FL

implementing synchronous updates is challenging, because

- devices might have limited computational resources
- evaluating the local loss may require data collection
- message passing over unreliable wireless links
- devices may spontaneously join or drop out (battery)

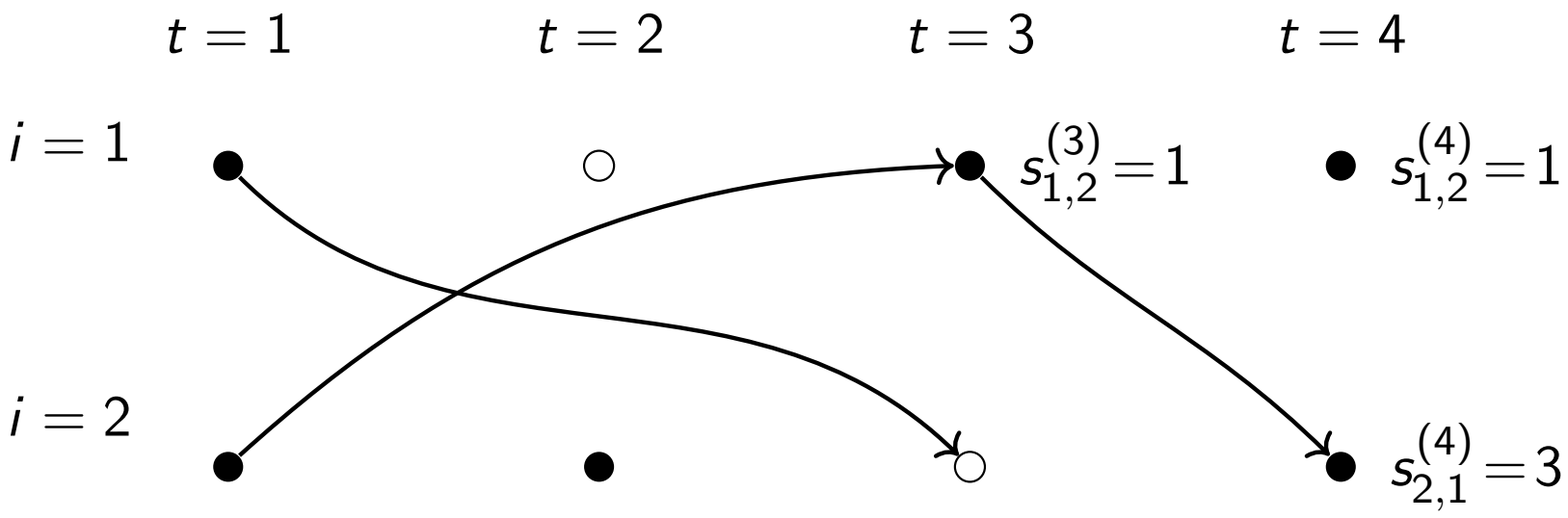
A Model for Asynchronous Algorithms

an asynchronous FL algorithm consists of update events

$$\mathbf{w}^{(i,t+1)} = \begin{cases} \mathcal{F}^{(i)}\left(\mathbf{w}^{(1,s_{i,1}^{(t)})}, \ldots, \mathbf{w}^{(n,s_{i,n}^{(t)})}\right) & \text{for } t \in \mathcal{U}^{(i)} \\ \mathbf{w}^{(i,t)} & \text{otherwise} \end{cases}$$

- iteration index t enumerates update events
- node i active only during events $t \in \mathcal{U}^{(i)}$
- at $t \in \mathcal{U}^{(i)}$, node i uses out-dated $\mathbf{w}^{(i',s_{i,i'}^{(t)})}$ (staleness)
- **delay** $t - s_{i,i'}^{(t)}$ from i' to i during update event $t \in \mathcal{U}^{(i)}$

Execution of Asynchronous FL



- FL network with two connected nodes $i = 1, 2$
- filled circle at time t indicates that node is active, i.e.

$$\mathcal{U}^{(1)} = \{1, 3, 4\} \text{ , and } \mathcal{U}^{(2)} = \{1, 2, 4\}$$

Example: Asynchronous FedGD

consider GD update operators

$$\mathcal{F}^{(i)}\left(\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}\right) = \mathbf{w}^{(i)} - \eta\left[\nabla L_i\left(\mathbf{w}^{(i)}\right) + 2\alpha\sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'}\left(\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right)\right].$$

asynchronous variant of FedGD is then

$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta\left[\nabla L_i\left(\mathbf{w}^{(i,t)}\right) + 2\alpha\sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'}\left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',s_{i,i'}^{(t)})}\right)\right],$$

for $t \in \mathcal{U}^{(i)}$, and $\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)}$ otherwise.

Python demo

Totally Asynchronous Algorithms

a totally asynchronous FL algorithm

$$\mathbf{w}^{(i,t+1)} = \begin{cases} \mathcal{F}^{(i)}\left(\mathbf{w}^{(1,s_{i,1}^{(t)})}, \ldots, \mathbf{w}^{(n,s_{i,n}^{(t)})}\right) & \text{for } t \in \mathcal{U}^{(i)} \\ \mathbf{w}^{(i,t)} & \text{otherwise.} \end{cases}$$

"works" under the following minimal assumptions:

- for each $i = 1, \ldots, n$, "wake-up" set $\mathcal{U}^{(i)}$ is infinite
- time instants $s_{i,i'}^{(t)}$ are unbounded,

$$\lim_{\substack{t \to \infty \\ t \in \mathcal{U}^{(i)}}} s_{i,i'}^{(t)} = \infty.$$

---

see Ch. 6 of D. Bertsekas, J. Tsitsiklis, "Parallel and Distributed Computation: Numerical Methods," 2015.

Partially Asynchronous Algorithms

a partially asynchronous FL algorithm

$$\mathbf{w}^{(i,t+1)} = \begin{cases} \mathcal{F}^{(i)}\left(\mathbf{w}^{(1,s_{i,i'}^{(t)})}, \ldots, \mathbf{w}^{(n,s_{i,n}^{(t)})}\right) & \text{for } t \in \mathcal{U}^{(i)} \\ \mathbf{w}^{(i,t)} & \text{otherwise.} \end{cases}$$

works as long as

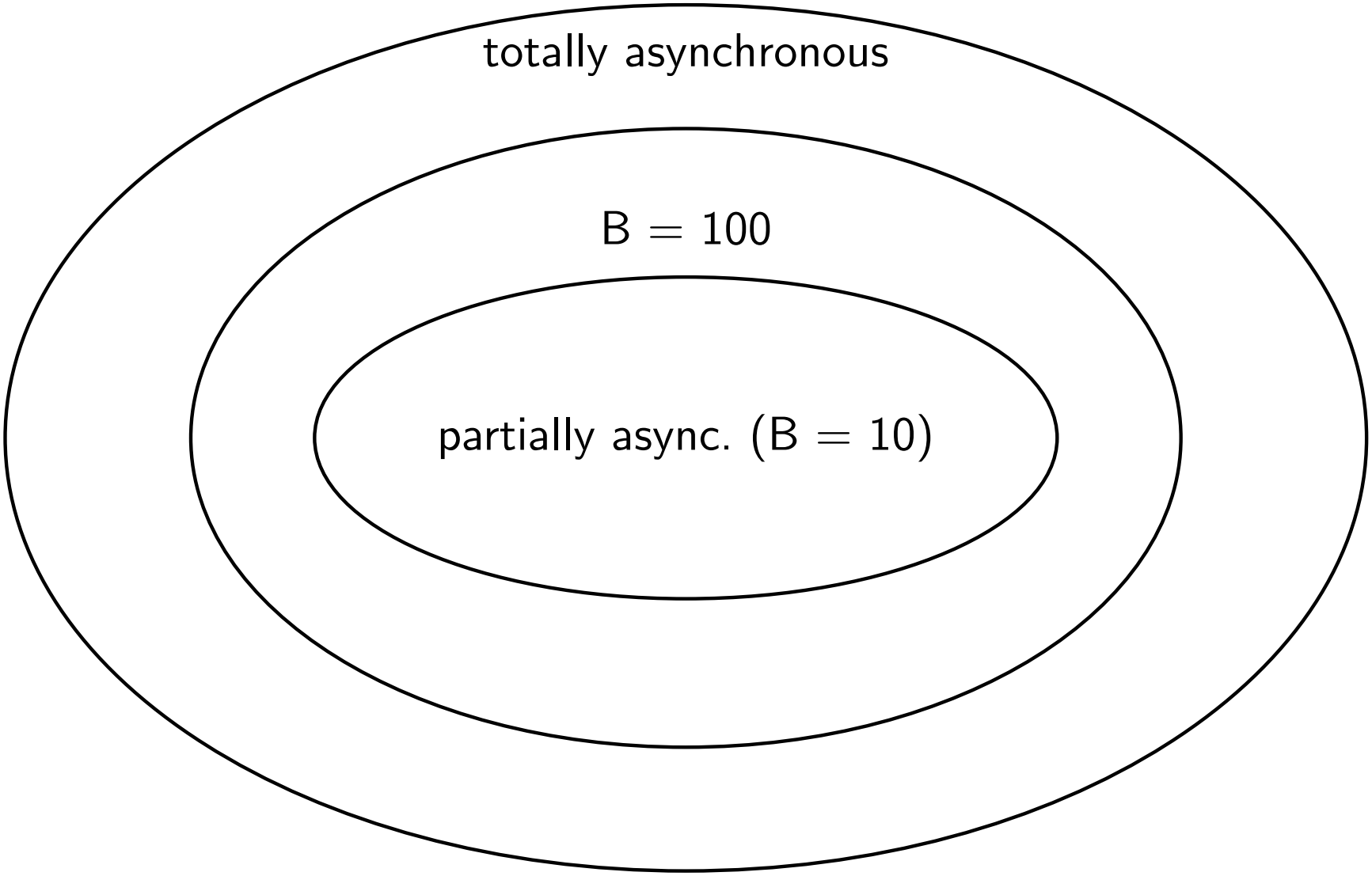
- $\{t, \ldots, t+B-1\} \cap \mathcal{U}^{(i)} \neq \emptyset$ for each $t \in \mathbb{N}$, and
- bounded delay $t - s_{i,i'}^{(t)} \leq B$ for all $t \in \mathcal{U}^{(i)}$

with a prescribed maximum delay $B \geq 1$

---

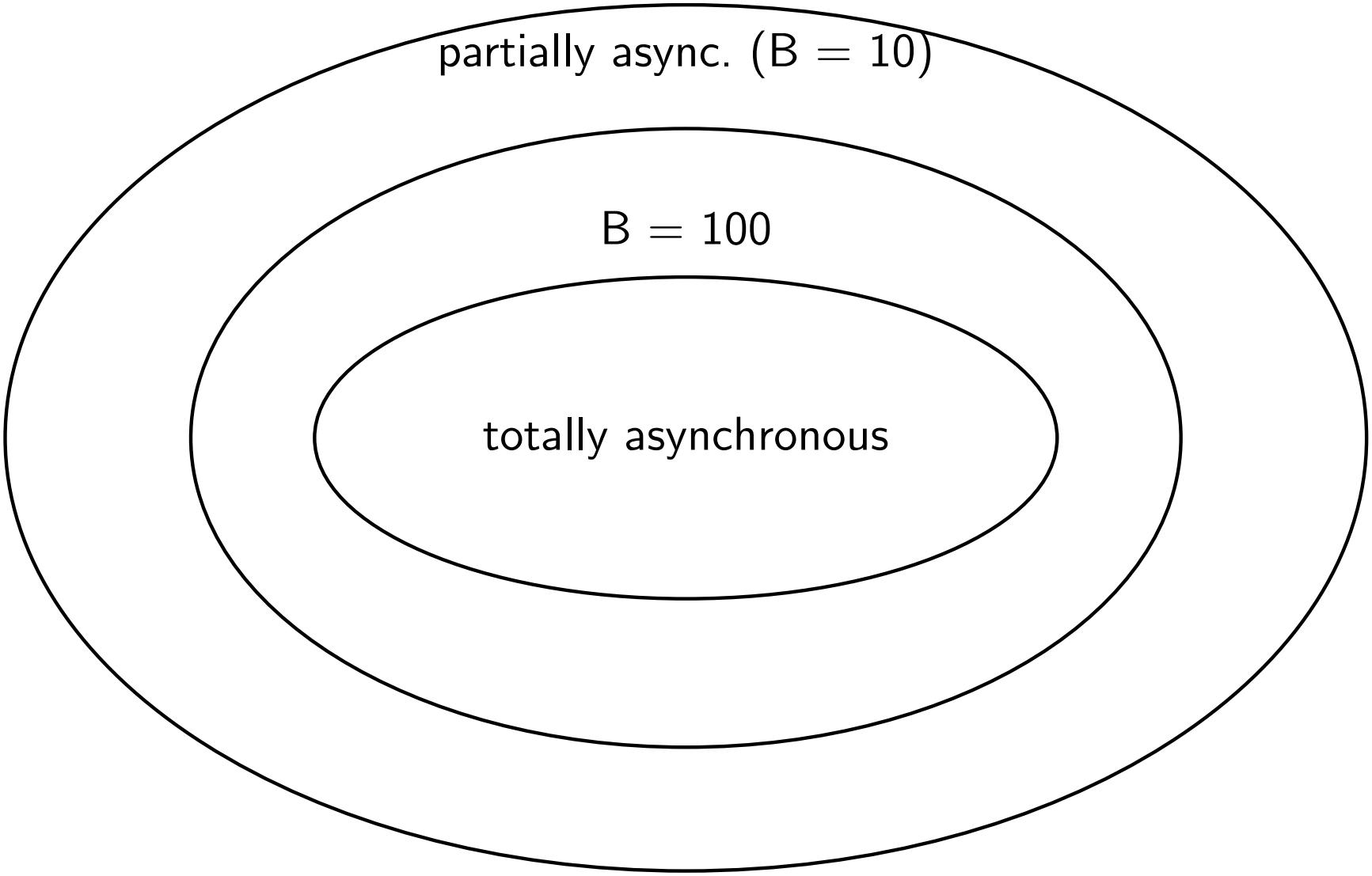
see Ch. 7 of D. Bertsekas, J. Tsitsiklis, "Parallel and Distributed Computation: Numerical Methods," 2015.

Hierarchy of Asynchronous Computers



maintaining a sufficiently synchronous execution of local model updates requires (painful) engineering efforts, e.g., to ensure small delay B

Hierarchy of FL Algorithms



totally asynchronous FL algorithms require proper design choices for local loss, local models and connectivity of FL network

When Does it Work ?

asynchronous FL algorithm is characterized by:

- the update operators $\mathcal{F}^{(i)}$ for $i = 1, \ldots, n$
- the update events $\mathcal{U}^{(i)}$, for $i = 1, \ldots, n$
- the delays $t - s_{i,i'}^{(t)}$ for $i, i' \in \mathcal{V}$, $t \in \mathcal{U}^{(i)}$

what are sufficient conditions on these characteristics such that $\mathbf{w}^{(i,t)}$ converges to a solution of some GTVMin instance

Pseudo-Contractions w.r.t. Block-Maximum Norm

- consider model parameters $\mathbf{w} = \left(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(n)}\right)$
- define the block-maximum norm

$$\|\mathbf{w}\|_\infty := \max_{i=1,\ldots,n} \left\|\mathbf{w}^{(i)}\right\|_i$$

joint operator $\mathcal{F} = \left(\mathcal{F}^{(1)}, \ldots, \mathcal{F}^{(n)}\right)$ is pseudo-contraction if

$$\|\mathcal{F}\mathbf{w} - \widehat{\mathbf{w}}\|_\infty \leq \kappa \|\mathbf{w} - \widehat{\mathbf{w}}\|_\infty$$

with fixed point $\widehat{\mathbf{w}} = \mathcal{F}\widehat{\mathbf{w}}$ and some $\kappa \in [0, 1)$

A Convergence Result

Consider an asynchronous FL algorithm using $\mathcal{F}$ with $\kappa < 1$. Then,

- it is totally asynchronous, and
- it is partially asynchronous with

$$\left\|\mathbf{w}^{(t)} - \widehat{\mathbf{w}}\right\|_\infty \leq \kappa^{\frac{k}{2B+1}} \left\|\mathbf{w}^{(0)} - \widehat{\mathbf{w}}\right\|_\infty.$$

$\Rightarrow$ convergence is faster for smaller κ and smaller B !

how can we ensure small κ and small B?

See Thm 23 and 24 of H. R. Feyzmahdavian and M. Johansson, "Asynchronous Iterations in Optimization: New Sequence Results and Sharper Algorithmic Guarantees," JMLR, 2023

Wrap Up

- FL as fixed-point iteration $\mathbf{w}^{(t+1)} = \mathcal{F}\mathbf{w}^{(t)}$.
- fixed points of $\mathcal{F}$ solve GTVMin
- convergence depends on contraction properties of $\mathcal{F}$
- asynchronous implementation possible if $\mathcal{F}$ is contraction in block-maximum norm
- asynchrony slows down convergence